First Test – Web Development Fundamentals

Time: 95 minutes **Total questions:** 60 **Rules:**

1. No AI tools, no copying from the internet. You may not open documentation during the test.
2. Answer in your own words. Short answers should be 1–3 sentences unless asked otherwise.
3. For coding questions, plain JavaScript/TypeScript or JSX is fine. Small syntax mistakes are OK. We are checking your understanding.
4. If you don't know an answer, write "I don't know" and move on. An honest blank is better than a guess.

Question types: **[MCQ]** multiple choice (pick one) · **[Short]** short answer · **[Code]** read or write code

---

Section A – Git (9 questions)

1. [MCQ] What does git clone <url> do? a) Creates an empty repository b) Copies a remote repository to your computer, including its history c) Uploads your local code to GitHub d) Creates a new branch
Ans. b) Copies a remote repository to your computer, including its history

2. [Short] What is the difference between git fetch and git pull?
Ans. git fetch downloads the latest changes from the remote repository without merging them into your local branches. git pull downloads the changes and automatically merges them into your currently checked-out branch.

3. [Short] What is the staging area? Which command moves changes into it?
Ans. The staging area is an intermediate holding space where you prepare files and changes before committing them. The Command is git add (File name).

4. [MCQ] Which command creates a new branch called cart-page and switches to it? a) git branch cart-page --go b) git switch -c cart-page c) git merge cart-page d) git init cart-page
Ans. b) git switch -c cart-page

5. [Short] What is a merge conflict, and what are the steps to resolve one?
Ans. A merge conflict happens when Git cannot Automatically resolve difference in code between two branches being merged. To resolve it : 1. Open the conflict file and edit the conflict markers to keep the desired code; 2. stage the resolved file using git add. 3. Complete the merge with git commit.

6. [Short] Why must you never commit a .env file? How do you stop Git from tracking it?
Ans. An .env file contains sensitive information like database credentials and API secrets, Committing it exposes them publicly if the repository is shared or public.
Using .gitignore I can stop Git from tracking it.

7. [Short] Write a good commit message for a change that adds a total price to the cart.
Ans. feat(cart) : calculate and display total price.

8. [Short] What is the difference between git revert and git reset --hard? Which one is safe to use on a shared branch?
Ans. git revert creates a new commit the undoes the changes of a previous history. git reset --hard completely destroys all commits and changes after a specific point by rewriting history. git revert is safe to use on a shared branch because it doesn't rewrite shared history.

9. [Short] What is a pull request, and why do teams use them instead of pushing straight to main?
Ans. A pull request is a proposal to merge changes from one branch into another in GitHub.
Teams use them too enable code reviews, run automated tests and prevent bugs or unauthorized code from going straight to main branch.

Section B – Web fundamentals (8 questions)

10. [Short] What is the difference between frontend and backend? Give one example of work done by each.
Ans. Frontend is the user-facing part of an application that runs in the browser like a navigation bar.
Backend is the server-side logic, database interaction and API handling like handling payments.
11. [MCQ] Which of these runs in the user's browser? a) A MongoDB query b) A React button's onClick handler c) A database migration d) Reading a server-only secret from process.env
Ans. b) A React button's onClick handler

12. [Short] List the main steps from typing https://example.com in the browser to seeing the page.
Ans. 1. Browser resolves example.com to an IP address via DNS.
2. Browser established a TCP connection and TLS handshake with the server. 3
. Browser sends a HTTP Get request.
4. Server processes the request and returns an HTTP response containing HTML,CSS and js.
5. Browser parses the code and renders the page.

13. [MCQ] Which HTTP method is normally used to create a new resource (for example, a new order)?
a) GET b) POST c) DELETE d) HEAD
Ans. b) POST

14. [Short] What do these status codes mean: 200, 201, 400, 401, 403, 404, 500?
Ans. 200 : Request succeeded.
201 : Request succeeded and a new resource was created.
400 : server can not process the request due to client error.
401 : Authentication is required and has failed.
403 : Server understands the request, but refuses to authorise it.
404 : The requested resource could not be found.
500 : The server encountered an unexpected error.

15. [Short] What is CORS, and why might your React app get a CORS error when calling an API?
Ans. CORS is a browser mechanism that restricts web pages from requesting to a different domain than the one that served the web page.
A react app might get a CORS error when it runs on one origin and the requested data from an another origin without proper headers.

16. [MCQ] Where should a payment provider's **secret** key be stored? a) In a React component b) In localStorage c) In a server-side environment variable d) In a public GitHub repository
Ans. c) In a server-side environment variable

17. [Short] What is JSON, and why do APIs use it?
Ans. JSON is a lightweight, text-based data interchange format.
APIs use it because it is human readable language and supported by javaScript and modern languages.

Section C – React (10 questions)

18. [Short] What is the difference between props and state?
Ans. Props are passed down from a parent component and are read only with in child.
State is managed internally within a component and can change over time.

19. [MCQ] Which of these causes a React component to re-render? a) Changing a normal let variable inside the component b) Calling the setter from useState with a new value c) Writing console.log d) Changing a CSS file

Ans. b) Calling the setter from useState with a new value

20. [Code] This counter never updates on screen. Why? How do you fix it?

```
function Counter() {
  let count = 0;
  return <button onClick={() => count++}>{count}</button>;
}
```

Normal variables like Let count = 0 does not persist across renders and do not trigger React re-renders when updated.

Fix : Const [counter, setCounter] = useState(0);

return ,button onClick={ () => setCounter(count + 1)}>{count}</button>;

21. [Short] Why does React need a key prop when rendering a list? Why is using the array index as the key a bad idea when items can be added or removed?

React needs a key prop to uniquely identify elements in a list for efficient reconciliation. Using array indices as keys is bad because if items are inserted, deleted or reordered, it leads to bugs with component state and incorrect UI rendering.

22. [Short] What does useEffect do? When does useEffect(() => { ... }, []) run?

Ans. useEffect lets me perform side effects like data fetching in functional components.

When passed an empty dependency array [ ], it runs exactly once after the component mounts.

23. [Code] This component freezes the browser. Why?

```
function Products() {
  const [products, setProducts] = useState([]);
  useEffect(() => {
    fetch("/api/products")
      .then((res) => res.json())
      .then((data) => setProducts(data));
  });
  return <p>{products.length} products</p>;
}
```

Ans. The useEffect hook is missing a dependency array, so it runs after *every* render. Inside the effect, calling setProducts triggers a state update, causing a re-render, which triggers the effect again in an infinite loop.

24. [MCQ] What is a "controlled input" in React? a) An input whose value is stored in React state and updated with onChange b) An input that is disabled c) An input inside a <form> tag d) An input styled with Tailwind
Ans. a) An input whose value is stored in React state and updated with onChange

25. [Short] A child component has an "Add to cart" button. How does it tell the parent component that it was clicked?
Ans. The parent passes a callback function like onAdd as a prop to the child component. When the button is clicked, the child invokes this callback function.

26. [Short] Your product list and your header (which shows the cart count) both need the cart. Where should the cart state live, and why?
Ans. It should live in their closest common ancestor component (or global state) through "lifting state up", because React state flows downward and sibling components cannot share state directly without a common parent.

27. [MCQ] Which hook stores a value that persists between renders **without** causing a re-render when it changes? a) useState b) useEffect c) useRef d) useMemo
Ans. c) useRef

Section D – React + separate backend vs Next.js (8 questions)

28. [Short] What does Next.js give you that a plain React app (for example, made with Vite) does not?
Ans. Next.js provides built-in Server-Side Rendering, file-based routing, Server Components, and API route handlers out of the box, whereas plain React is purely a client-side library.

29. [Short] In an app built with React (frontend) + Express or FastAPI (backend) + MongoDB (database), describe how a product list gets from the database to the screen. Name each step.
Ans. 1. Browser requests the React app.
2. React app sends an HTTP GET request to the backend endpoint.
3. Backend queries MongoDB.
4. MongoDB returns raw documents to the backend.
5. Backend sends JSON response to React.
6. React updates state via useState and renders the product list.

30. [MCQ] In the Next.js App Router, components are by default: a) Client components b) Server components c) Static HTML files d) API routes
Ans. b) Server components

31. [Short] When do you need to add "use client" at the top of a Next.js component? Give two examples.
Ans. You need "use client" when a component requires browser-only features or interactivity.
Examples: 1. Using useState or useEffect hooks.
2. Handling browser events like onClick.

32. [Short] Explain CSR, SSR and static generation in one sentence each. Which one would you pick for a product detail page, and why?
Ans. CSR: The browser downloads minimal HTML and executes JS to render content.
SSR: The server generates the HTML page on every request and sends the fully rendered page.
Static Generation: HTML pages are generated at build time.
SSR for a product detail page to ensure fresh pricing and stock on every request.

33. [MCQ] The file app/products/[id]/page.tsx handles which URL? a) /products b) /products/42 c) /api/products d) /page/products
Ans. b) /products/42

34. [Short] What is a route handler (app/api/.../route.ts) in Next.js? Give one reason a team might still build a separate backend instead.
Ans. A route handler allows creating custom backend endpoints using Web Request/Response APIs inside a Next.js application.
Teams might build a separate backend for micro services, complex background tasks, or independent scalability.

35. [Short] Why can't a client component read process.env.STRIPE_SECRET_KEY? What does the NEXT_PUBLIC_ prefix do, and what should never use it?
Ans. Client components run in the browser, meaning any environment variables accessible there are visible in source code. The NEXT_PUBLIC_ prefix exposes a variable to the browser bundle; private keys like databases should never use it.

Section E – APIs (4 questions)

36. [Short] Design REST endpoints (method + path) for: listing products, getting one product, adding an item to the cart, removing an item from the cart.
Ans. Listing products: GET /api/products
Getting one product: GET /api/products/:id
Adding an item to the cart: POST /api/cart
Removing an item from the cart: DELETE /api/cart/:itemId

37. [Short] Your form already checks that the email is valid. Why must the backend check it again?
Ans. Client-side validation can easily be bypassed or manipulated via developer tools or direct API requests. The backend must validate all input to ensure data integrity and security.

38. [MCQ] For GET /api/products?search=rice&page=2, search and page are: a) Headers b) Query parameters c) Cookies d) Request body
Ans. b) Query parameters

39. [Code] Write a function getProducts() that calls GET /api/products with fetch, returns the JSON, and throws an error if the response is not OK.
Ans.
```
async function getProducts() {
  const res = await fetch('/api/products');
  if (!res.ok) {
    throw new Error('Failed to fetch products');
  }
  return res.json();
}
```

Section F – Authentication, sessions and cookies (9 questions)

40. [Short] What is the difference between authentication and authorization? Give an example of each from an online shop.
Ans. Authentication verifies *who* you are. Authorization determines *what* you are allowed to do.

41. [Short] Explain step by step how session-based login works, from submitting the login form to staying logged in on the next request.

Ans. 1. User submits credentials.

2. Server verifies them.

3. Server creates a session record and associates it with a session ID.

4. Server sends a Set-Cookie header with the session ID.

5. Browser automatically stores and sends the cookie back on subsequent requests.

42. [Short] What is a JWT? Give one advantage and one disadvantage compared to a server-side session.

Ans. A JWT is a compact, URL-safe token format digitally signed with user claims.

Advantage: Statelessness, like no database lookup required per request

Disadvantage: Harder to revoke instantly before expiration.

43. [Short] What is a cookie, and how does the browser use it?

Ans. A cookie is a small piece of data stored by the browser and automatically attached to HTTP requests for the same domain, used for session management and state.

44. [MCQ] Which cookie flag stops JavaScript in the page from reading the cookie? a) Secure b) HttpOnly c) SameSite d) Max-Age

Ans. b) HttpOnly

45. [Short] What do the Secure and SameSite flags do?

Ans. Secure ensures cookies are only sent over HTTPS.

SameSite controls whether cookies are sent with cross-site requests, protecting against attacks.

46. [Short] Why is storing a login token in localStorage considered risky? What attack is the concern?

Ans. localStorage is accessible via any JavaScript on the page, meaning an XSS vulnerability can allow malicious scripts to steal the token.

47. [Short] Why do we hash passwords instead of encrypting them or storing them as plain text?

Ans. Hashing is a one-way mathematical transformation; even if the database is leaked, passwords cannot be reversed back to plain text unlike encryption, which is reversible.

48. [MCQ] What is CSRF? a) An attacker injects JavaScript into your page b) Another website tricks the user's browser into sending a request to your site using the user's cookies c) An attacker guesses passwords many times d) A database query is changed by user input

Ans. b) Another website tricks the user's browser into sending a request to your site using the user's cookies

Section G – Databases (8 questions)

49. [Short] What is the main difference between a SQL database (like PostgreSQL) and MongoDB?
Ans. SQL databases are relational, structured, use tables with fixed schemas, and link data via foreign keys.
MongoDB is a NoSQL document database storing flexible, schema-less JSON-like documents.

50. [Short] Give one scenario where MongoDB is a better fit and one where a relational database is a better fit. Explain why.
Ans. **MongoDB:** Content management systems where attributes vary widely and schemas change frequently.
Relational DB: E-commerce systems where strong ACID compliance and complex multi-table joins are critical.

51. [Short] What is a database migration, and why do teams use migrations instead of changing the database by hand?
Ans. A migration is a version-controlled script that systematically applies schema changes.
Teams use them to ensure reproducibility and consistency across environments.

52. [MCQ] Supabase is built on top of: a) MongoDB b) MySQL c) PostgreSQL d) Firebase
Ans. c) PostgreSQL

53. [Short] What is an index, and how does it make search faster? Is there a cost?
Ans. An index is a data structure that speeds up data retrieval by avoiding full table scans.
Cost: It consumes extra storage and slows down write operations because the index must be updated.

54. [Short] Design the tables (or collections) for users, products and cart items. List the main fields and how they connect.
Ans. Users: id, name, email, password_hash
Products: id, name, price, stock
CartItems: id, user_id (FK to Users), product_id (FK to Products), quantity.

55. [Short] What are a primary key and a foreign key?
Ans. A primary key uniquely identifies a record in a table. A foreign key is a field in one table that references the primary key of another table, linking them.

56. [Short] What is a transaction? Why does it matter when a customer places an order and stock must go down?
Ans. A transaction executes a sequence of database operations as a single unit. It matters for orders because I must ensure that *both* the order is created and the product stock is decremented together, preventing partial failures like overselling.

Section H – Short coding (4 questions)

57. [Code] Write a function cartTotal(items) that returns the total price. Each item looks like { name: "Rice", price: 250, quantity: 2 }.
Ans.
```
function cartTotal(items) {
  return items.reduce((total, item) => total + (item.price * item.quantity), 0);
}
```
58. [Code] Write a React component ProductCard that receives name, price and onAdd as props, shows the name and price, and calls onAdd when an "Add to cart" button is clicked.
Ans.
```
function ProductCard({ name, price, onAdd }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>Price: ${price}</p>
      <button onClick={onAdd}>Add to cart</button>
    </div>
  );
```

59. [Code] Write a function searchProducts(products, term) that returns the products whose name contains term, ignoring upper and lower case.
Ans.
```
function searchProducts(products, term)
  const lowerTerm = term.toLowerCase()
  return products.filter(product =>
    product.name.toLowerCase().includes(lowerTerm)
  );
}
```

60. [Code] Write a React component SearchBox with a controlled text input. When the user types, it should call an onSearch(value) prop with the current text.

Ans.
```
import React, { useState } from 'react';
function SearchBox({ onSearch }) {
  const [query, setQuery] = useState('');
  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);
    onSearch(value);
  };
  retur (
    <input
      type="text"
      value={query}
      onChange={handleChange}
      placeholder="Search..."
    />
  );
}
```

End of test. Save your answers as firsttest-answers.md, push them to your learning-log repository, and share the link in Slack.